

Task 21:

Implement a CI/CD pipeline using Jenkins, GitHub Actions, or GitLab CI/CD. Automate code checkout, build, test, and deployment steps, and demonstrate continuous integration and continuous deployment for the application

Aim

To implement a **CI/CD pipeline** using a tool like **GitHub Actions** (or Jenkins) to automate **code checkout, build, testing, and deployment**, enabling continuous integration and continuous deployment.

Algorithm

1. Create a GitHub repository for the project.
2. Add project code (Spring Boot app).
3. Create a .github/workflows folder.
4. Create a workflow file (ci-cd.yml).
5. Define pipeline steps:
 - Checkout code
 - Setup Java
 - Build using Maven
 - Run tests
6. Push code to GitHub.
7. GitHub Actions automatically triggers workflow.
8. Monitor pipeline execution.
9. Verify build success/failure.
10. Deploy application (optional step).

Code Implementation

1. GitHub Actions Workflow File

github/workflows/ci-cd.yml

name: CI/CD Pipeline

on:

push:

branches: ["main"]

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout Code

uses: actions/checkout@v3

- name: Set up Java

uses: actions/setup-java@v3

with:

java-version: '17'

- name: Build with Maven

run: mvn clean install

- name: Run Tests

run: mvn test

Output:

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

Settings

Workflows

New workflow

All workflows

Audit

All workflows

Showing runs from all workflows

Filter workflow runs

2 workflow runs

Event ▾

Status ▾

Branch ▾

Actor ▾

✔ Fix sayHello() to return the correct value

Audit #2: Commit 986a2ac pushed by alexedwards

main

2 minutes ago ...

34s

✖ Initial commit

Audit #1: Commit 91f141a pushed by alexedwards

main

17 minutes ago ...

33s

Result

CI/CD pipeline successfully implemented Automatic build and test execution on code push
Continuous integration achieved Improved development efficiency

